

## This session we're going to cover:

- Tuples
- Sets

## تاپل پایتون

تاپل (Tuple)

مجموعه‌ای شبیه به لیست در پایتون است. تفاوت اصلی این است که پس از ایجاد یک تاپل، نمی‌توان آن را تغییر داد

## ایجاد یک تاپل در پایتون

ما با قرار دادن عناصر درون پرانتز () یک تاپل ایجاد می‌کنیم. برای مثال

```
In [2]: numbers = (1, 2, -5)
print(numbers)
```

(1, 2, -5)

ما همچنین می‌توانیم با استفاده از سازنده‌ی `tuple()` یک تاپل ایجاد کنیم. برای مثال

```
In [4]: tuple_constructor = tuple(('Jack', 'Maria', 'David'))
print(tuple_constructor)
```

('Jack', 'Maria', 'David')

در اینجا انواع مختلف تاپل‌هایی (tuples) که می‌توانیم در پایتون ایجاد کنیم، آمده است

```
In [6]: # create an empty tuple
empty_tuple = ()
print(empty_tuple)

# tuple of string types
names = ('James', 'Jack', 'Eva')
print (names)

# tuple of float types
float_values = (1.2, 3.4, 2.1)
print(float_values)

# tuple including string and integer
mixed_tuple = (2, 'Hello', 'Python')
print(mixed_tuple)
```

```
()  
( 'James', 'Jack', 'Eva' )  
(1.2, 3.4, 2.1)  
(2, 'Hello', 'Python')
```

## ویژگی‌های تاپل

- مرتب شده - آن‌ها ترتیب عناصر را حفظ می‌کنند
- تغییرناپذیر - پس از ایجاد، قابل تغییر نیستند
- اجازه دادن به مقادیر تکراری - می‌توانند شامل مقادیر تکراری باشند

## دسترسی به عناصر تاپل

هر عضو در یک تاپل (tuple) با عددی مرتبط است که اندیس (index) نامیده می‌شود

اندیس همیشه از ۰ شروع می‌شود؛ بدین معنا که اولین عضو تاپل در اندیس ۰، دومین عضو در اندیس ۱ و به همین ترتیب قرار دارد

```
In [8]: languages = ('Python', 'Swift', 'C++')
```

```
# access the first item  
print(languages[0])
```

```
# access the third item  
print(languages[2])
```

Python

C++

## تاپل قابل تغییر نیست

تاپل‌های پایتون تغییرناپذیر (immutable) هستند

ما نمی‌توانیم آیتم‌های یک تاپل را اضافه، تغییر یا حذف کنیم

اگر سعی کنیم یک تاپل را تغییر دهیم، با خطا مواجه خواهیم شد. برای مثال

```
In [9]: cars = ('BMW', 'Tesla', 'Ford', 'Toyota')
```

```
# trying to modify a tuple  
cars[0] = 'Nissan'      # error  
print(cars)
```

```

-----
TypeError                                Traceback (most recent call last)
Cell In[9], line 4
      1 cars = ('BMW', 'Tesla', 'Ford', 'Toyota')
      2
      3 # trying to modify a tuple
----> 4 cars[0] = 'Nissan'      # error
      5 print(cars)

TypeError: 'tuple' object does not support item assignment

```

## طول تاپل در پایتون

تابع `len()`

برای یافتن تعداد عناصر موجود در یک تاپل استفاده می‌کنیم. برای مثال

```

In [11]: cars = ('BMW', 'Tesla', 'Ford', 'Toyota')
         print('Total Items:', len(cars))

```

Total Items: 4

## متدهای تاپل در پایتون

- Python Tuple `count()`

متد `count()` تعداد دفعات تکرار عنصر مشخص شده در تاپل را برمی‌گرداند

- Python Tuple `index()`

متد `index()` اندیس عنصر مشخص شده در تاپل را برمی‌گرداند

```

In [12]: # tuple of vowels
         vowels = ('a', 'e', 'i', 'o', 'i', 'u')

         # counts the number of i's in the tuple
         count = vowels.count('i')

         print(count)

```

2

```

In [13]: # tuple containing vowels
         vowels = ('a', 'e', 'i', 'o', 'u')

         # index of 'e' in vowels
         index = vowels.index('e')

         print(index)

```

1

به طور کلی، زمانی باید از تاپل‌ها استفاده کنید که نیاز دارید :

- تضمین یکپارچگی داده‌ها: تاپل‌ها تغییرناپذیر هستند؛ بدین معنا که پس از ایجاد، نمی‌توانید عناصر آن‌ها را تغییر دهید. این ویژگی تغییرناپذیری، ثبات داده‌ها را تضمین کرده و باعث می‌شود مقادیر موجود در تاپل بدون تغییر باقی بمانند
- کاهش مصرف حافظه: تاپل‌ها در مقایسه با لیست‌ها سربار حافظه کمتری دارند، زیرا مقدار ثابتی از حافظه را به خود اختصاص می‌دهند. این ویژگی به‌ویژه هنگام کار با مجموعه‌های بزرگ داده یا در محیط‌هایی با محدودیت حافظه، مزیت محسوب می‌شود
- بهبود عملکرد: تاپل‌ها معمولاً از نظر ایجاد، پیمایش و دسترسی به عناصر، کارآمدتر از لیست‌ها هستند. این ویژگی می‌تواند منجر به بهبود عملکرد شود، به‌ویژه هنگام کار با مجموعه‌داده‌های حجیم

## مجموعه‌ها در پایتون

مجموعه (Set)

به معنای گردایه‌ای از داده‌های منحصر به فرد است؛ بدین معنا که عناصر درون یک مجموعه نمی‌توانند تکراری باشند

برای مثال، اگر نیاز به ذخیره اطلاعات مربوط به شماره‌های دانشجویی داشته باشیم، استفاده از مجموعه مناسب است؛ زیرا شماره‌های دانشجویی نمی‌توانند تکراری باشند

## ایجاد یک مجموعه (Set) در پایتون

در پایتون، ما مجموعه‌ها را با قرار دادن تمام عناصر درون آکولاد `{ }` و جدا کردن آن‌ها با ویرگول ایجاد می‌کنیم

می‌تواند شامل هر تعداد عضو باشد و این اعضا می‌توانند از انواع مختلفی (مانند عدد یک مجموعه (Set) صحیح، عدد اعشاری، تاپل، رشته و غیره) باشند

اما یک مجموعه نمی‌تواند (mutable) نظیر لیست‌ها، مجموعه‌ها یا دیکشنری‌ها باشد شامل عناصر تغییرپذیر

```
In [16]: # create a set of integer type
student_id = {112, 114, 116, 118, 115}
print('Student ID:', student_id)

# create a set of string type
vowel_letters = {'a', 'e', 'i', 'o', 'u'}
print('Vowel Letters:', vowel_letters)

# create a set of mixed data types
mixed_set = {'Hello', 101, -2, 'Bye'}
print('Set of mixed data types:', mixed_set)
```

Student ID: {112, 114, 115, 116, 118}  
Vowel Letters: {'o', 'u', 'a', 'i', 'e'}  
Set of mixed data types: {101, 'Hello', 'Bye', -2}

در مثال بالا، با قرار دادن تمام عناصر درون آکولاد {}، انواع مختلفی از مجموعه‌ها را ایجاد کرده‌ایم

نکته: هنگام اجرای این کد، ممکن است خروجی را با ترتیبی متفاوت دریافت کنید؛ زیرا مجموعه ترتیب خاصی ندارد

## ایجاد یک مجموعه خالی در پایتون

ایجاد یک مجموعه خالی کمی چالش‌برانگیز است؛ چرا که استفاده از آکولادهای خالی {} در پایتون، یک دیکشنری خالی ایجاد می‌کند

برای ایجاد یک مجموعه بدون `set()` بدون هیچ آرگومانی استفاده می‌کنیم. برای مثال هیچ عضوی، از تابع

```
In [17]: # create an empty set
empty_set = set()

# create an empty dictionary
empty_dictionary = { }

# check data type of empty_set
print('Data type of empty_set:', type(empty_set))

# check data type of dictionary_set
print('Data type of empty_dictionary:', type(empty_dictionary))
```

Data type of empty\_set: <class 'set'>  
Data type of empty\_dictionary: <class 'dict'>

## اقدام تکراری در یک مجموعه

```
In [20]: numbers = {2, 4, 6, 6, 2, 8}
print(numbers)
```

{8, 2, 4, 6}

در اینجا می‌بینیم که هیچ عنصر تکراری‌ای در مجموعه وجود ندارد، زیرا یک مجموعه نمی‌تواند شامل عناصر تکراری باشد

## افزودن و به‌روزرسانی عناصر مجموعه (Set) در پایتون

مجموعه‌ها تغییرپذیر هستند؛ با این حال، از آنجا که فاقد ترتیب هستند، اندیس‌گذاری در آن‌ها معنایی ندارد

ما نمی‌توانیم با استفاده از نمایه‌سازی یا برش‌دهی به یکی از عناصر مجموعه دسترسی پیدا کنیم یا آن را تغییر دهیم؛ چرا که نوع داده‌ای مجموعه از این قابلیت پشتیبانی نمی‌کند

## افزودن عناصر به یک مجموعه در پایتون

از متد `add()`

در پایتون، برای افزودن یک عضو به یک مجموعه (set) استفاده می‌کنیم. برای مثال

```
In [22]: numbers = {21, 34, 54, 12}

print('Initial Set:', numbers)

# using add() method
numbers.add(32)

print('Updated Set:', numbers)
```

Initial Set: {34, 12, 21, 54}  
Updated Set: {32, 34, 12, 21, 54}

## به‌روزرسانی مجموعه (Set) در پایتون

متد `update()`

با عناصر موجود در سایر انواع مجموعه‌ها (مانند لیست‌ها، تاپل‌ها، مجموعه‌ها و غیره) برای به‌روزرسانی یک مجموعه (set)، استفاده می‌شود. برای مثال

```
In [23]: companies = {'Lacoste', 'Ralph Lauren'}
tech_companies = ['apple', 'google', 'apple']

# using update() method
companies.update(tech_companies)

print(companies)
```

{'Ralph Lauren', 'google', 'apple', 'Lacoste'}

## حذف یک عنصر از مجموعه

ما از متد `discard()`

برای حذف عنصر مشخص شده از یک مجموعه استفاده می‌کنیم. برای مثال

```
In [24]: languages = {'Swift', 'Java', 'Python'}

print('Initial Set:', languages)

# remove 'Java' from a set
```

```
removedValue = languages.discard('Java')
```

```
print('Set after remove():', languages)
```

Initial Set: {'Java', 'Swift', 'Python'}

Set after remove(): {'Swift', 'Python'}

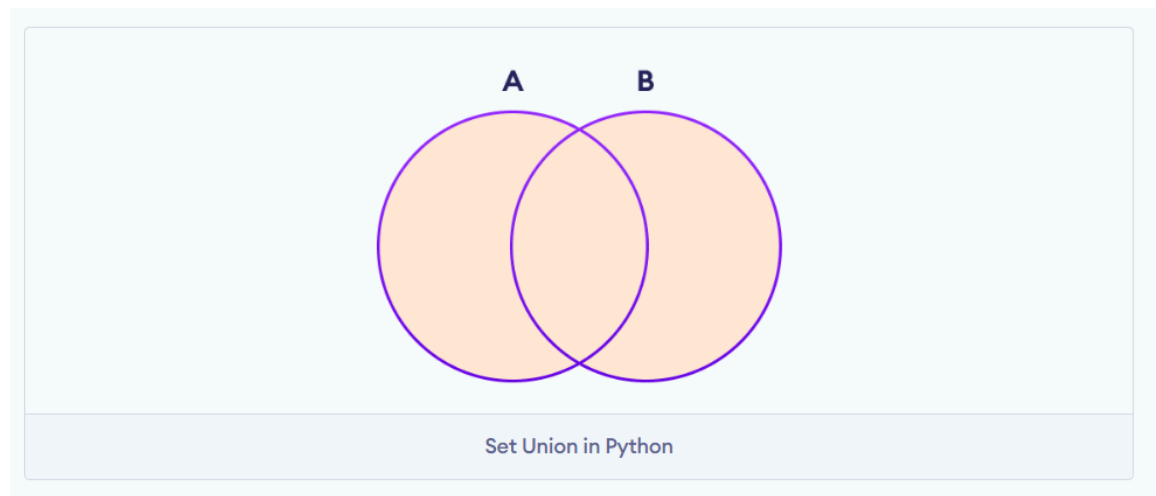
## عملیات مجموعه در پایتون

نوع داده‌ای Set

در پایتون، متدهای داخلی متنوعی را برای انجام عملیات ریاضی مجموعه‌ها — مانند اجتماع، اشتراک، تفاضل و تفاضل متقارن — فراهم می‌کند

## اجتماع دو مجموعه

اجتماع دو مجموعه A و B شامل تمام اعضای مجموعه‌های A و B است



ما از `union()` برای انجام عملیات اتحاد مجموعه‌ها استفاده می‌کنیم. برای مثال عملگر `|` یا متد

```
In [25]: # first set
A = {1, 3, 5}

# second set
B = {0, 2, 4}

# perform union operation using |
print('Union using |:', A | B)

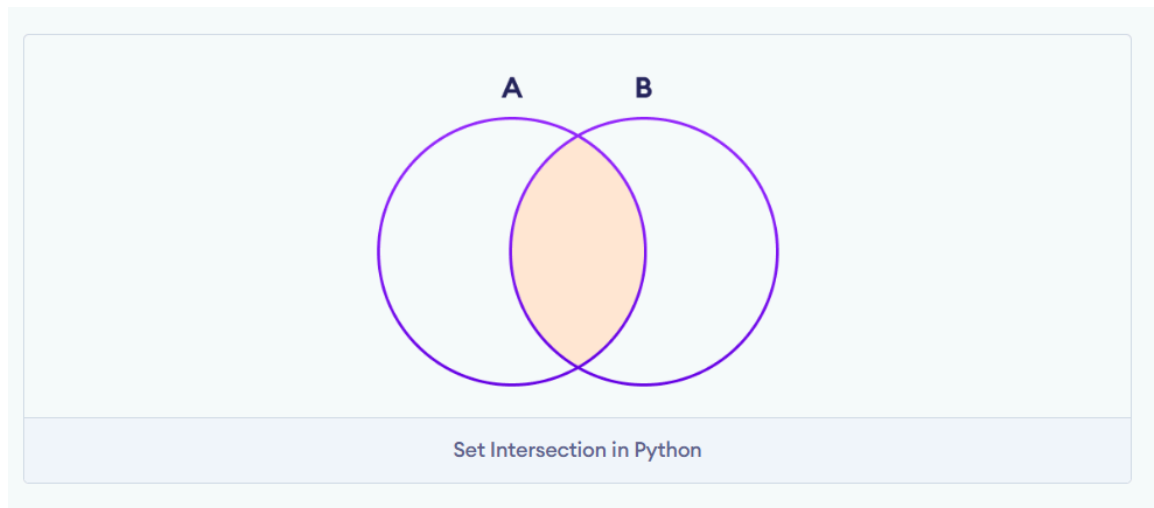
# perform union operation using union()
print('Union using union():', A.union(B))
```

Union using |: {0, 1, 2, 3, 4, 5}

Union using union(): {0, 1, 2, 3, 4, 5}

## اشتراک مجموعه‌ها

اشتراک دو مجموعه A و B شامل عناصر مشترک بین مجموعه A و B است



از عملگر & یا متد intersection()

در پایتون برای انجام عملیات اشتراک مجموعه‌ها استفاده می‌کنیم برای مثال

```
In [27]: # first set
A = {1, 3, 5}

# second set
B = {1, 2, 3}

# perform intersection operation using &
print('Intersection using &:', A & B)

# perform intersection operation using intersection()
print('Intersection using intersection():', A.intersection(B))
```

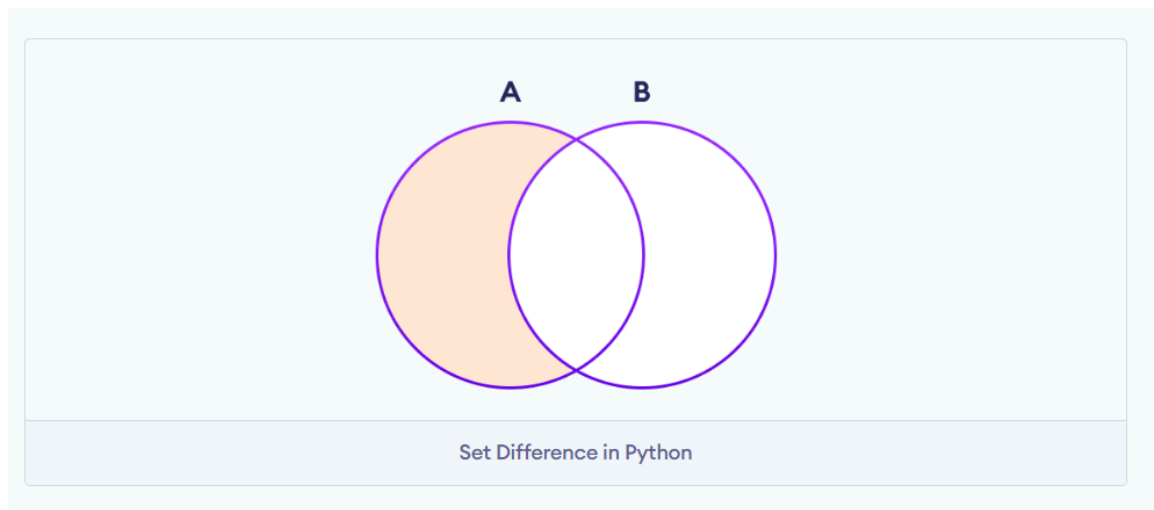
Intersection using &: {1, 3}

Intersection using intersection(): {1, 3}

## تفاوت بین دو مجموعه

تفاضل دو A و B شامل عناصری از مجموعه A است که در مجموعه B وجود ندارند  
مجموعه





از عملگر - یا متد `difference()`

برای محاسبه تفاضل بین دو مجموعه استفاده میکنیم

```
In [29]: # first set
A = {2, 3, 5}

# second set
B = {1, 2, 6}

# perform difference operation using &
print('Difference using &:', A - B)

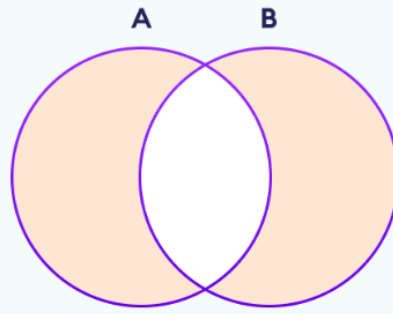
# perform difference operation using difference()
print('Difference using difference():', A.difference(B))
```

Difference using &: {3, 5}

Difference using difference(): {3, 5}

## تفاضل متقارن مجموعه‌ها

تفاضل متقارن بین دو A و B شامل تمام اعضای A و B به جز اعضای مشترک آنهاست  
مجموعه



Set Symmetric Difference in Python

در پایتون، ما از عملگر `^` یا `symmetric_difference()` استفاده می‌کنیم.

برای انجام تفاضل‌های متقارن بین دو مجموعه استفاده می‌کنیم. برای مثال یا متد

```
In [30]: # first set
A = {2, 3, 5}

# second set
B = {1, 2, 6}

# perform difference operation using &
print('using ^:', A ^ B)

# using symmetric_difference()
print('using symmetric_difference():', A.symmetric_difference(B))
```

using ^: {1, 3, 5, 6}

using symmetric\_difference(): {1, 3, 5, 6}

## توابع دخی Set

| Function                 | Description                                                                                                    |
|--------------------------|----------------------------------------------------------------------------------------------------------------|
| <code>all()</code>       | Returns <code>True</code> if all elements of the set are true (or if the set is empty).                        |
| <code>any()</code>       | Returns <code>True</code> if any element of the set is true. If the set is empty, returns <code>False</code> . |
| <code>enumerate()</code> | Returns an enumerate object. It contains the index and value for all the items of the set as a pair.           |
| <code>len()</code>       | Returns the length (the number of items) in the set.                                                           |
| <code>max()</code>       | Returns the largest item in the set.                                                                           |
| <code>min()</code>       | Returns the smallest item in the set.                                                                          |
| <code>sorted()</code>    | Returns a new sorted list from elements in the set (does not sort the set itself).                             |
| <code>sum()</code>       | Returns the sum of all elements in the set.                                                                    |

Table 3-1. Python set operations

| Function                                      | Alternative syntax      | Description                                                                               |
|-----------------------------------------------|-------------------------|-------------------------------------------------------------------------------------------|
| <code>a.add(x)</code>                         | N/A                     | Add element x to set a                                                                    |
| <code>a.clear()</code>                        | N/A                     | Reset set a to an empty state, discarding all of its elements                             |
| <code>a.remove(x)</code>                      | N/A                     | Remove element x from set a                                                               |
| <code>a.pop()</code>                          | N/A                     | Remove an arbitrary element from set a, raising <code>KeyError</code> if the set is empty |
| <code>a.union(b)</code>                       | <code>a   b</code>      | All of the unique elements in a and b                                                     |
| <code>a.update(b)</code>                      | <code>a  = b</code>     | Set the contents of a to be the union of the elements in a and b                          |
| <code>a.intersection(b)</code>                | <code>a &amp; b</code>  | All of the elements in <i>both</i> a and b                                                |
| <code>a.intersection_update(b)</code>         | <code>a &amp;= b</code> | Set the contents of a to be the intersection of the elements in a and b                   |
| <code>a.difference(b)</code>                  | <code>a - b</code>      | The elements in a that are not in b                                                       |
| <code>a.difference_update(b)</code>           | <code>a -= b</code>     | Set a to the elements in a that are not in b                                              |
| <code>a.symmetric_difference(b)</code>        | <code>a ^ b</code>      | All of the elements in either a or b but <i>not both</i>                                  |
| <code>a.symmetric_difference_update(b)</code> | <code>a ^= b</code>     | Set a to contain the elements in either a or b but <i>not both</i>                        |
| <code>a.issubset(b)</code>                    | <code>&lt;=</code>      | True if the elements of a are all contained in b                                          |
| <code>a.issuperset(b)</code>                  | <code>&gt;=</code>      | True if the elements of b are all contained in a                                          |
| <code>a.isdisjoint(b)</code>                  | N/A                     | True if a and b have no elements in common                                                |

داستان: آقای بنر یک کلاس برنامه‌نویسی پایتون با ۱۰ دانش‌آموز برگزار می‌کند. او هر روز صبح، ۱۵ دقیقه را صرف بررسی حضور و غیاب با استفاده از یک دفتر ثبت کاغذی می‌کند. گاهی اوقات او به اشتباه نام دانش‌آموز دیگری را علامت می‌زند، نام‌ها را تکرار می‌نویسد یا فراموش می‌کند که غایبان را بشمارد

```
In [16]: # Permanent class list (Tuple)
class_list = ( "Alice", "Bob", "Charlie", "David", "Emma", "Frank", "Grace", "Henry"

# Students who attended today (Set)
present = {"Alice", "Charlie", "Emma", "Grace", "Henry"}

# Convert tuple to set for calculations
all_students = set(class_list)

# Find absent students
absent = all_students - present

# Display results
print("==== Attendance Report =====")

print("\nClass List:")
print(class_list)

print("\nPresent Students:")
print(present)

print("\nAbsent Students:")
print(absent)

print("\nTotal Students:", len(class_list))
print("Present:", len(present))
print("Absent:", len(absent))

attendance_percentage = (len(present) / len(class_list)) * 100

print("\nAttendance Percentage:", attendance_percentage, "%")
```

==== Attendance Report =====

Class List:

('Alice', 'Bob', 'Charlie', 'David', 'Emma', 'Frank', 'Grace', 'Henry')

Present Students:

{'Charlie', 'Henry', 'Grace', 'Alice', 'Emma'}

Absent Students:

{'Bob', 'David', 'Frank'}

Total Students: 8

Present: 5

Absent: 3

Attendance Percentage: 62.5 %